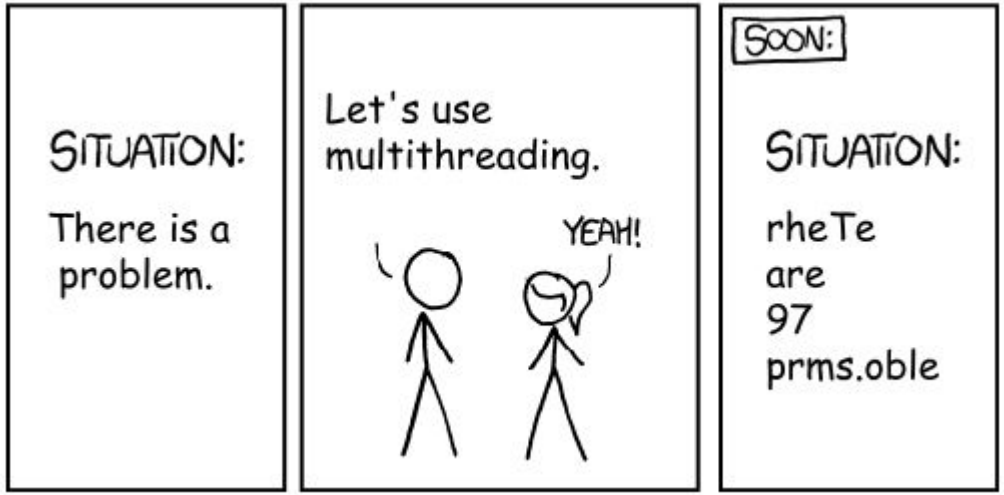


Software Engineering

Threading

What Actually Is Java Code?

What does `java MyProgram` actually do?

Tells the operating system to start a **process**

The process runs the JVM (Java Virtual Machine)

Fun fact: the JVM is an **API** between compiled java code and the actual OS

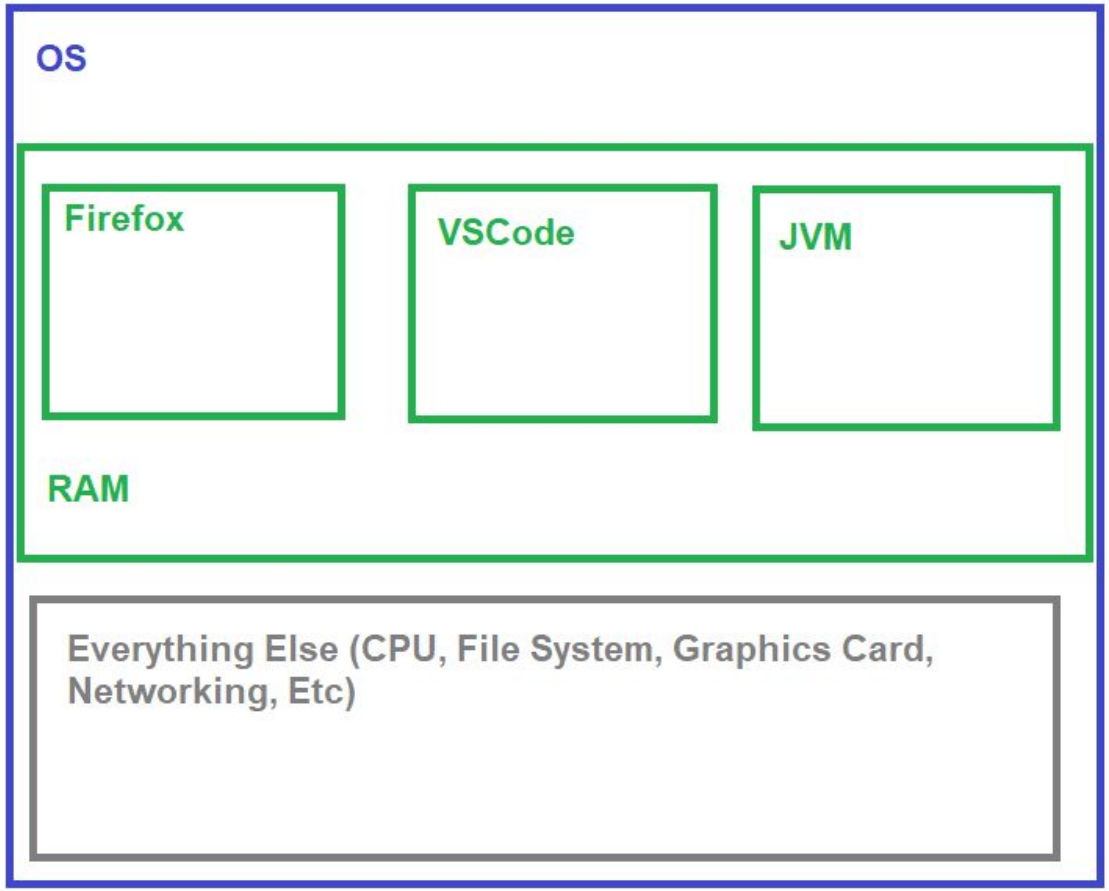
The JVM calls the `public static void main(String[] args)` that we know and love on the **main thread**

Process vs Thread

A **process** has an OS-level chunk of memory

No process is allowed to mess with another's memory

All other resources (File System, etc) are shared between processes

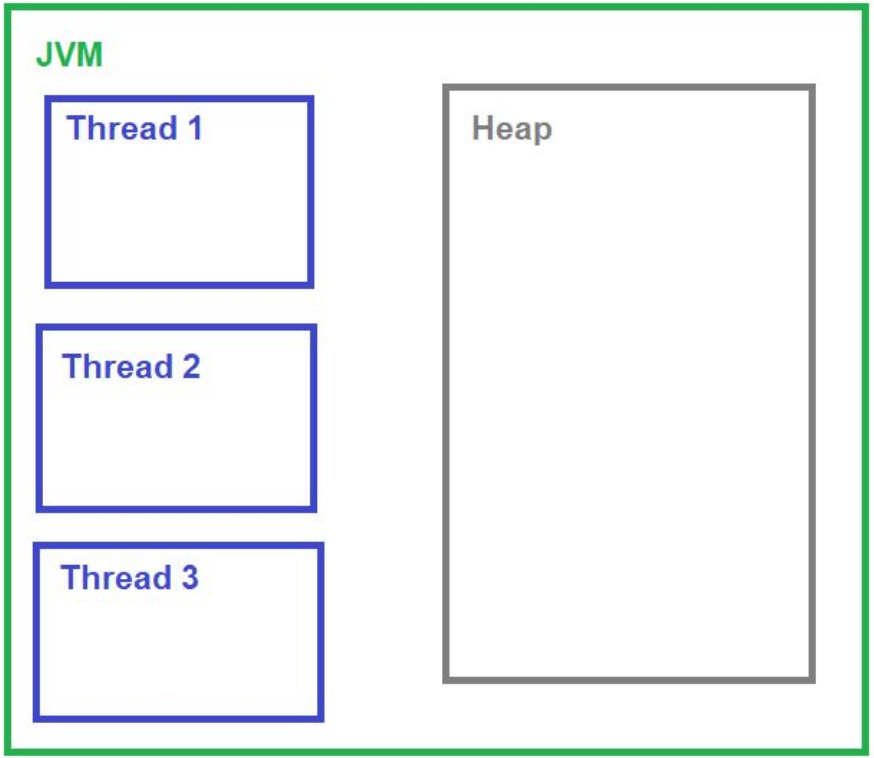


Process vs Thread

A **thread** is contained within a process

It has its own memory (**stack**) and shared memory (**heap**)

It can only use **one** core of one CPU at a time (linearly follows instructions)



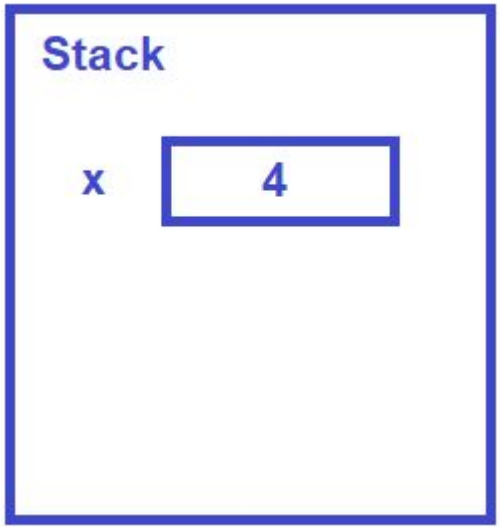
Stack: (Mostly) Primitive Types

Data on the stack is a fixed, known size.

Local variable primitive types: int, boolean, long, etc

Simple Local Variable: On the Stack

```
int x = 4;
```

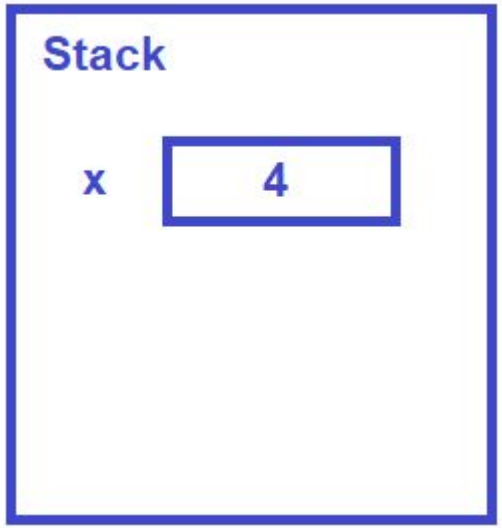


Why is it called the "Stack"?

Variables get pushed onto the stack, then popped when you leave a **scope** (usually a curly-brace section)

Ex:

```
int x = 4;
while (x < 5) {
    int y = x;
    x = y + 1;
}
```

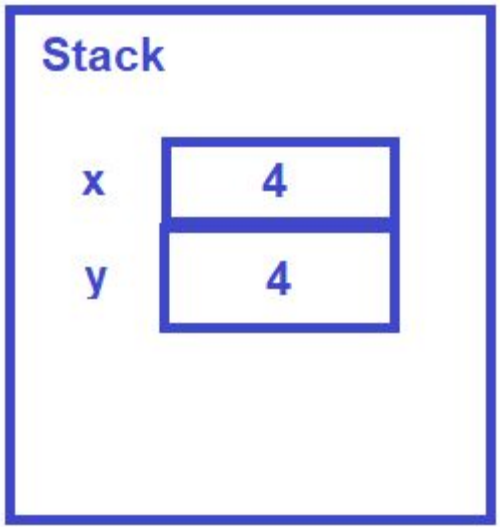


Why is it called the "Stack"?

Here, y is now **in scope**

Ex:

```
int x = 4;
while (x < 5) {
    int y = x;
    x = y + 1;
}
```

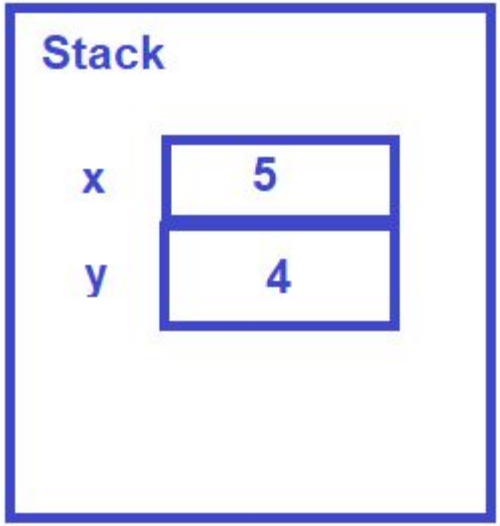


Why is it called the "Stack"?

We're still within the inner braces, so the stack is unchanged (just the values change)

Ex:

```
int x = 4;
while (x < 5) {
    int y = x;
    x = y + 1;
}
return x;
```

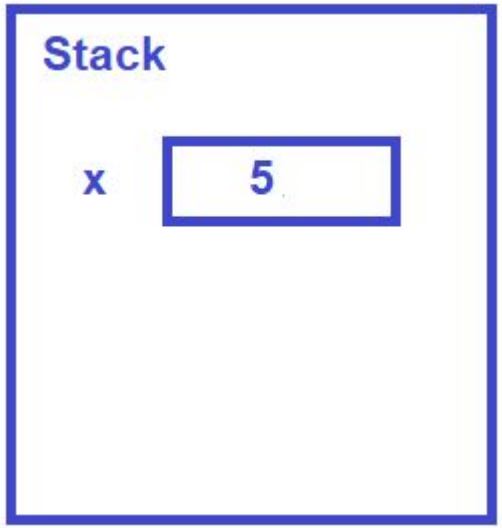


Why is it called the "Stack"?

Now y is out of scope, so it gets popped off the stack. This is why scopes are nested!

Ex:

```
int x = 4;
while (x < 5) {
    int y = x;
    x = y + 1;
}
return x;
```



What Else is On The Stack?

Program Control Metadata:

- Instruction Pointer - what line of code to go to next
- Return values
- Parameters

Stack being tightly controlled ⇒ very difficult to accidentally overwrite this data

When people say Java is a “secure” language, this is what they mean

What's on The Heap?

Almost everything!

- All fields within a class
- Anything to the right of a `new`
- String literals

Pointers

Claymation, pointers, and hilarity:
https://www.youtube.com/watch?v=vm5MNP7pn5g

Primitive type:

```
int x = 4;
```

In Memory:

In Program:
"Hey computer, when I say **x**, I mean the 4 bytes located at 0x44ED"

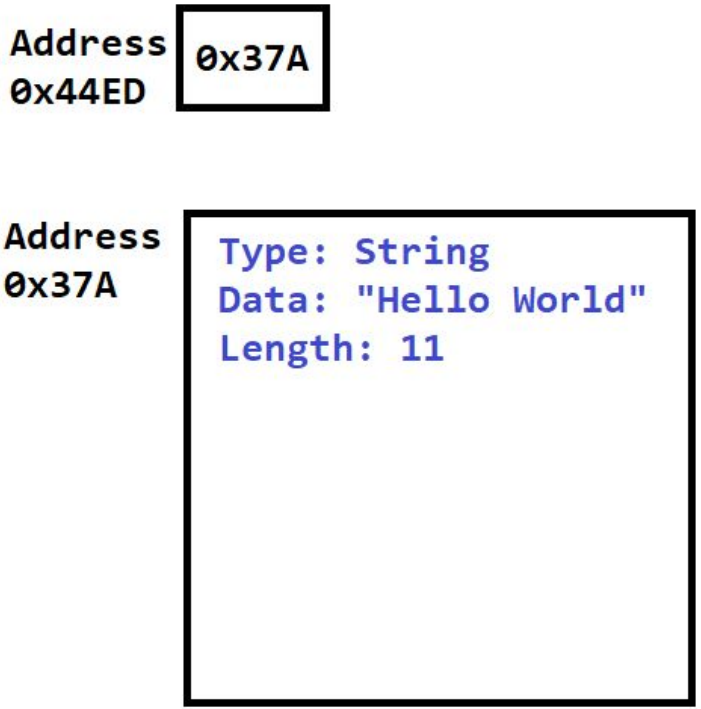
Pointers

Object type:

```
String s = "Hello World";
```

In Program:
"Hey computer, when I say **s**, I mean the 8 bytes located at 0x44ED"

In Memory



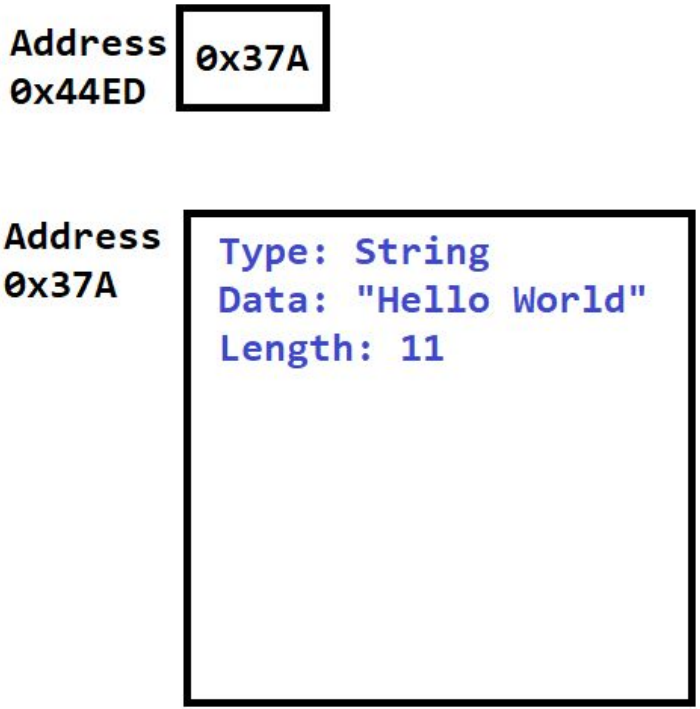
Pointers

Object type:

```
String s = "Hello World";
s.length();
```

In Program:
"Hey computer, when I say ., start with the 8 bytes at the variables address, then 'follow the pointer' and use the data located at the memory address stored in those 8 bytes"

In Memory



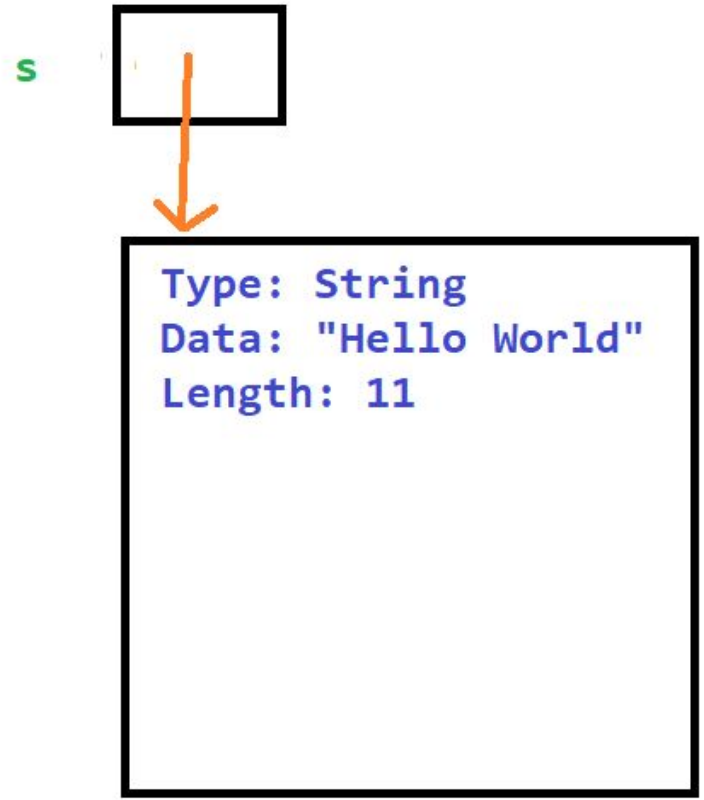
Pointers

Object type:

```
String s = "Hello World";
```

The pointer “s” is on the **stack**, the data pointed to is on the heap

Shorthand



Objects on The Heap

```
public class FileSystem {
  private final File root;
  private int numFiles;

  public FileSystem() {
    root = new File("/home");
    numFiles = 1;
  }
}
```

What does

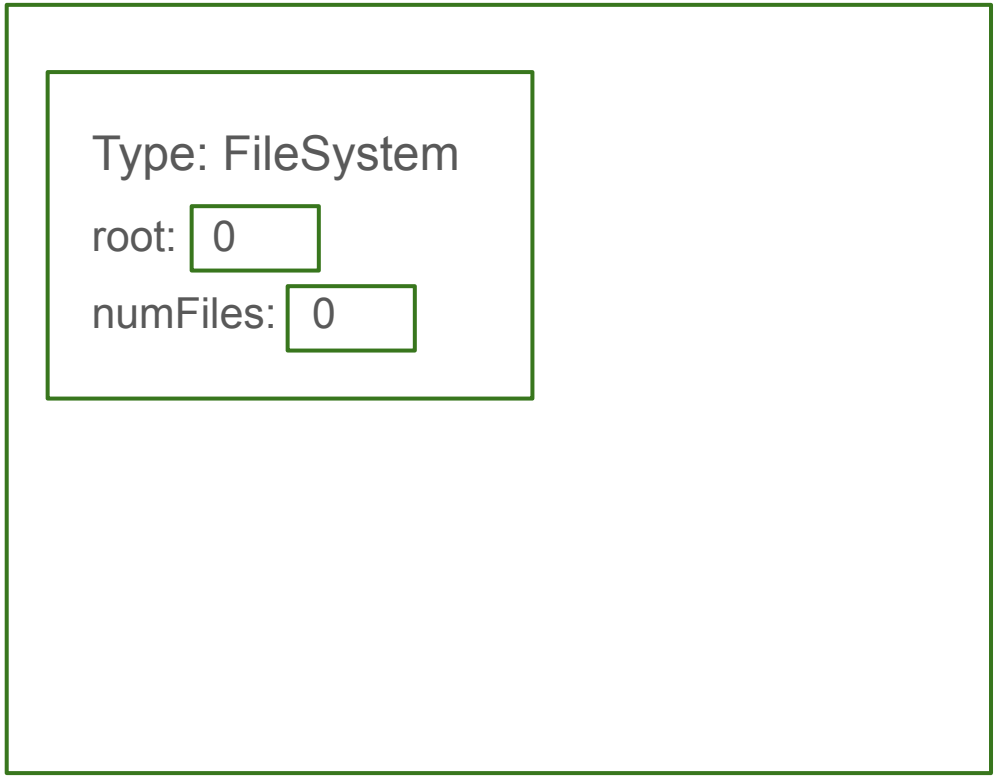
```
FileSystem fs = new FileSystem();
```

do?

Objects on The Heap

```
public class FileSystem {
  private final File root;
  private int numFiles;

  public FileSystem() {
    root = new File("/home");
    numFiles = 1;
  }
}
```

Step 1: Create a "shell" FileSystem object on the heap

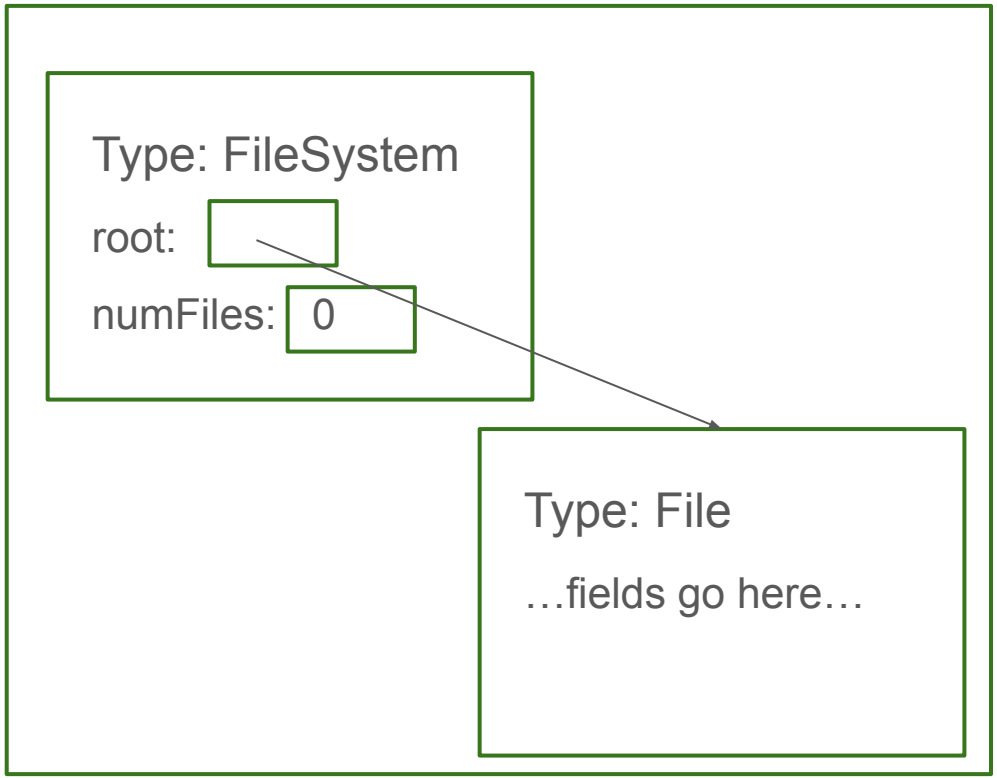


Objects on The Heap

```
public class FileSystem {
  private final File root;
  private int numFiles;

  public FileSystem() {
    root = new File("/home");
    numFiles = 1;
  }
}
```

Step 2: Call the constructor:

```
root = new File("/home");
```



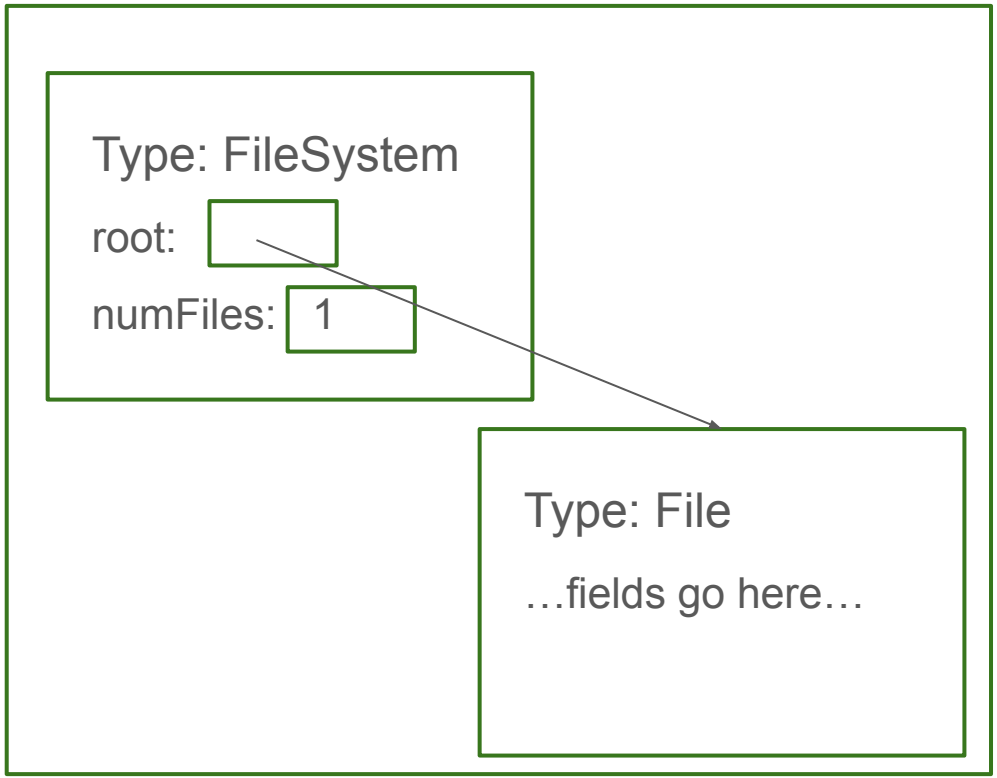
Objects on The Heap

```
public class FileSystem {
  private final File root;
  private int numFiles;

  public FileSystem() {
    root = new File("/home");
    numFiles = 1;
  }
}
```

Step 3: Call the constructor:

```
numFiles = 1;
```

Notice that the fields are on the **heap**, even if they have a primitive type



Objects on The Heap

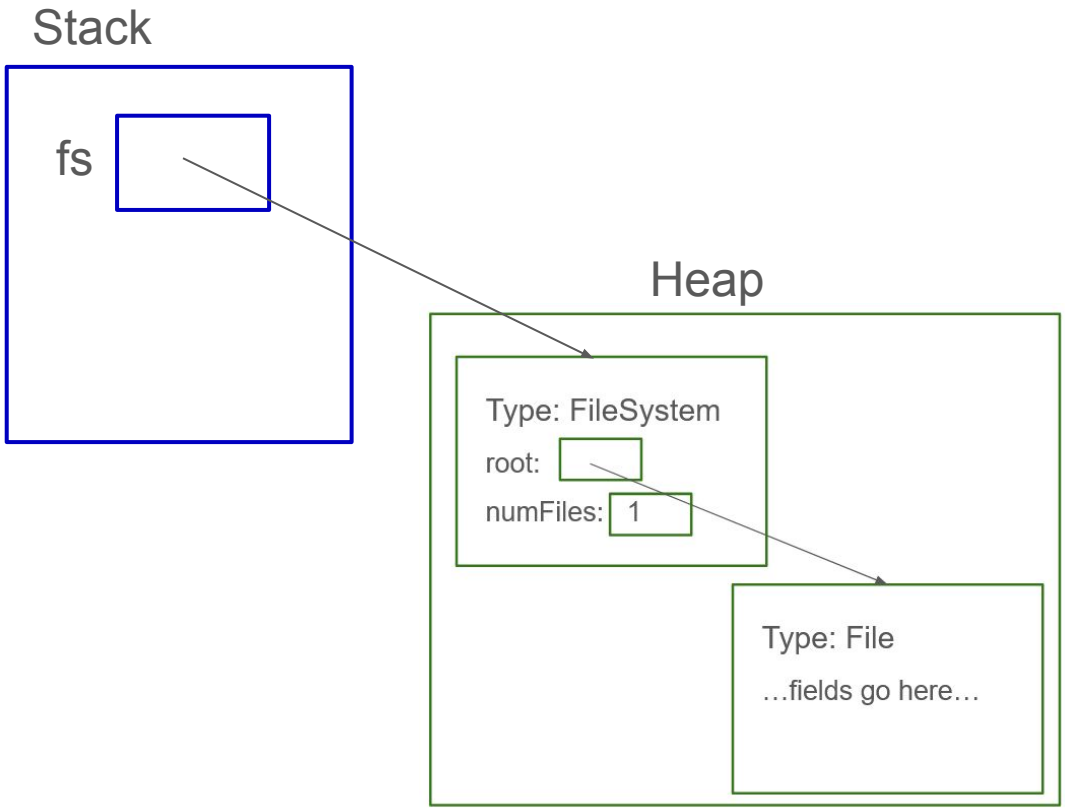
```
public class FileSystem {
  private final File root;
  private int numFiles;

  public FileSystem() {
    root = new File("/home");
    numFiles = 1;
  }
}
```

Step 4: Set up the local variable

```
FileSystem fs = new FileSystem();
```

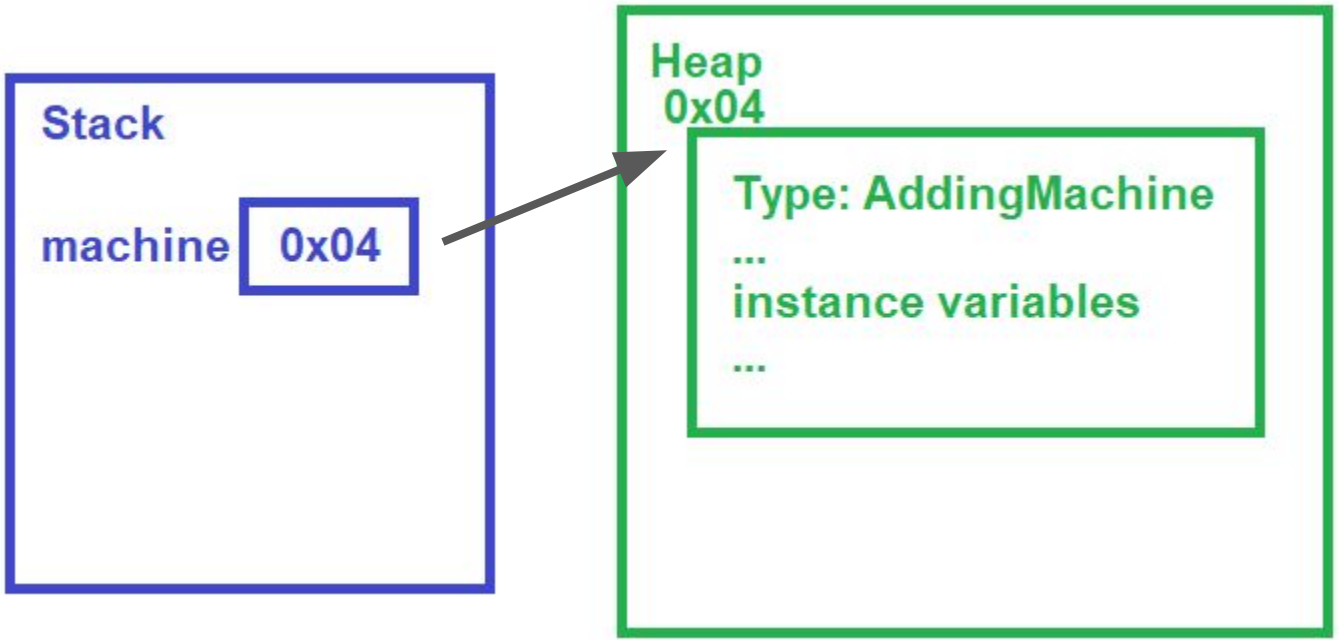
Note that Java is a **strongly typed** language - each blob o' bytes knows what type it is

- Not true in languages like C or Python



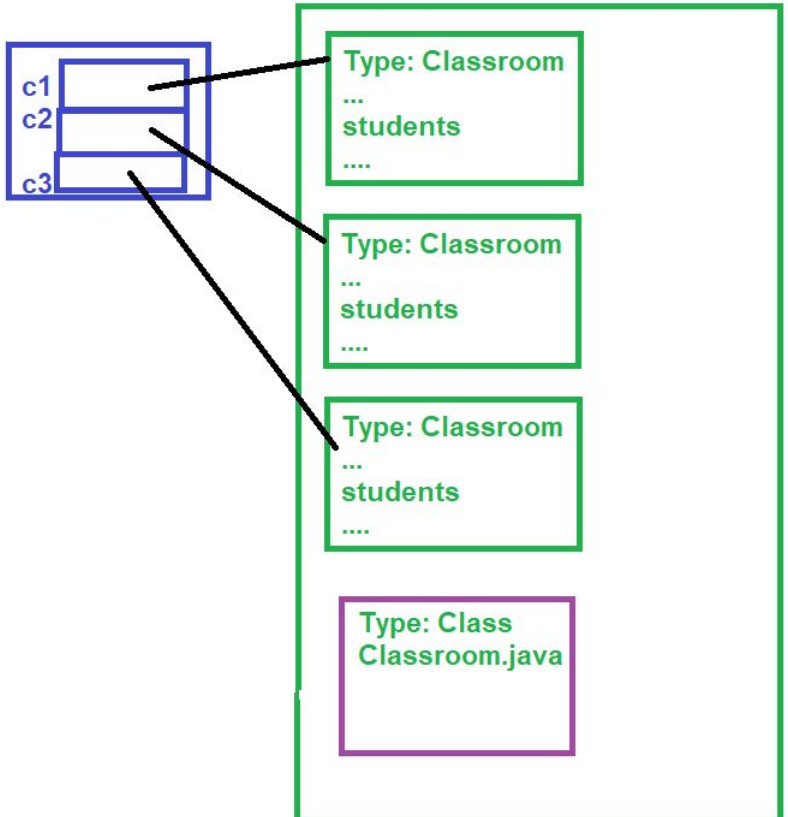
Practical Applications of Deconstruction


```
AddingMachine machine = new AddingMachine();
```



Distinction Between "class" and "instance"

```
Classroom c1 = new Classroom();
Classroom c2 = new Classroom();
Classroom c3 = new Classroom();
```



What About Arrays?

Contiguous block **on the heap**

- The array knows its length (big difference in Java)

Exercise: Match the code with the memory layout

1. Check out the ThreadingExercises repo (link on Brightspace)
2. Work through MemoryMatching.pdf

Parameters: Pointers vs Pointees

In Java:

- All primitive types are passed by **value**: a copy of the data is made, modifying it doesn't change the original
- All object types are passed by **reference**: a copy of the **pointer** is made, but not the data, so modifying the data changes the original

Alt interpretation: all parameters are passed by making a copy of what's on the **stack**, but not copying what's on the the **heap**

What Does This Print?

```
public static void main(String[] args) {

    int one = 1;
    int two = increment(one);

    System.out.println(one);
    System.out.println(two);

}

/**  Adds 1 to the value */
public static int increment(int value) {
    value++;
    return value;
}
```

What About This?

```
public static void main(String[] args) {

    IntObject one = new IntObject(1);
    IntObject two = increment(one);

    System.out.println(one.value);
    System.out.println(two.value);

}

/**  Adds 1 to the value */
public static IntObject
       increment(IntObject intObject) {
    intObject.value++;
    return new IntObject(intObject.value);
}
```

```
public class IntObject {

    public int value;

    public IntObject(int initialValue) {
        value = initialValue;
    }

}
```

Your Turn! (Pointers, Heap, Stack recap)

1. Look at the exercise1 package in ThreadingExercises
2. The program is supposed to print out a week's worth of dates

Ex:

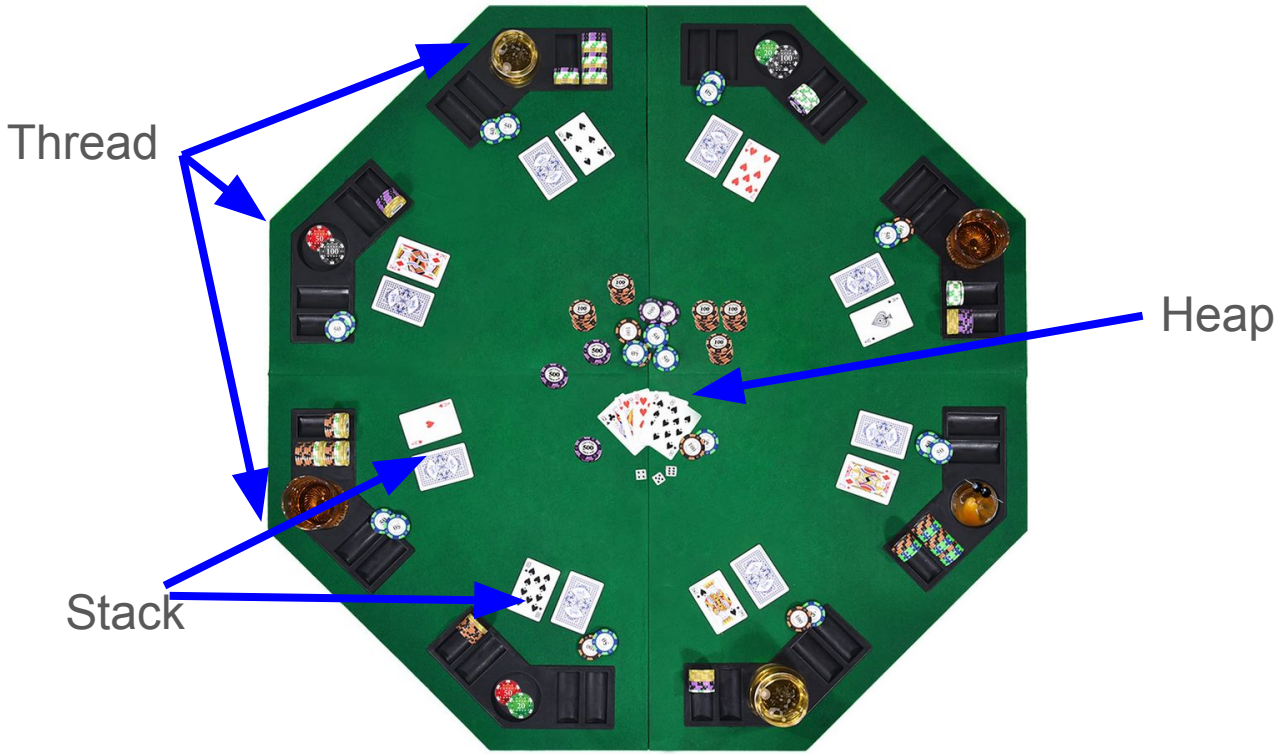
```
Info for the day 9/25/2024 (the next day will be 9/26/2024)
Info for the day 9/26/2024 (the next day will be 9/27/2024)
```

etc

But it's not quite right - first identify what's wrong with the output

3. Now, fix the code to print the days as it’s supposed to

Threads



Threads

Each thread follows the same rules, but the actual behavior can differ:

Player 1 plays a yellow 9 ⇒ Player 2 plays a 4

or

Player 1 plays a green 3 ⇒ Player 2 plays a 5

Player 1

Player 2

Threads in Java

Don't ever use low-level threading classes: Thread, wait, notify

- multi-threaded code has enough bugs as it is, don't make your life harder

Instead, use the `java.util.concurrent` library (comes with the jre, much like java Collections)

To **create** a thread: use a `ThreadPool`, generally wrapped by an `ExecutorService`

To tell the thread **what it should do**, use a `Callable<T>` (supports lambda syntax)

To check on **what the thread did**, use a `Future<T>` and call get()

Threads in Java

Use the `java.util.concurrent` library

- Your IDE will figure out these imports for you!

```
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class PlayGame {

    public void playGame() {
        Callable<Void> player = () -> {
            while (!gameIsOver()) {
                playCard();
            }
            return null;
        };

        ExecutorService threadPool = Executors.newCachedThreadPool();
        int nPlayers = 2;
        for (int i = 0 ; i < nPlayers; i++) {
            threadPool.submit(player);
        }
    }
```

Threads in Java

The **Callable** determines what each thread will do.

No return value ⇒ use the `Void` type

`null` is the only valid value for something of type `Void` ⇒ always `return null` for this type

```
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class PlayGame {

    public void playGame() {
        Callable<Void> player = () -> {
            while (!gameIsOver()) {
                playCard();
            }
            return null;
        };

        ExecutorService threadPool = Executors.newCachedThreadPool();
        int nPlayers = 2;
        for (int i = 0 ; i < nPlayers; i++) {
            threadPool.submit(player);
        }
    }
```

Threads in Java

Use `Executors` to get an `ExecutorService` to run the threads

`newCachedThreadPool` is a good default

Other options:

- give all the threads a name
- use a fixed size pool
- define what should happen if too many things are submitted to the service
- many more

```
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class PlayGame {

    public void playGame() {
        Callable<Void> player = () -> {
            while (!gameIsOver()) {
                playCard();
            }
            return null;
        };

        ExecutorService threadPool = Executors.newCachedThreadPool();
        int nPlayers = 2;
        for (int i = 0 ; i < nPlayers; i++) {
            threadPool.submit(player);
        }
    }
```

Threads in Java

Use **submit** to actually start a thread

This code is missing something important!

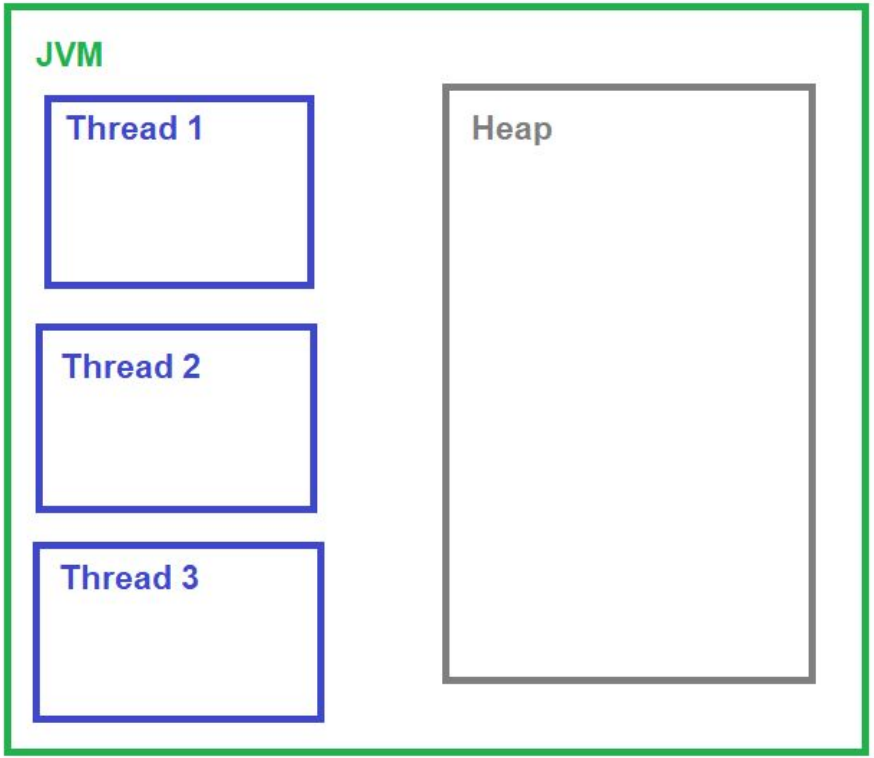
- Handling exceptions
- Waiting for the threads to finish running

```
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class PlayGame {

    public void playGame() {
        Callable<Void> player = () -> {
            while (!gameIsOver()) {
                playCard();
            }
            return null;
        };

        ExecutorService threadPool = Executors.newCachedThreadPool();
        int nPlayers = 2;
        for (int i = 0 ; i < nPlayers; i++) {
            threadPool.submit(player);
        }
    }
```

Exceptions & Threads

An **exception** will halt the thread it happens on, but doesn't cross thread boundaries on its own

Use **Future** to find out about those exceptions



Threads in Java

Use **Future** to do two things:

- Wait for a thread to finish, and return its result
- Check for an Exception

.get() is a **blocking** call - you may not want to call it right away

```
    ExecutorService threadPool = Executors.newCachedThreadPool();
    int nPlayers = 2;
    for (int i = 0 ; i < nPlayers; i++) {
        threadPool.submit(player);
    }
}
```

```
List<Future<?>> exceptionChecker = new ArrayList<>();
for (int i = 0 ; i < nPlayers; i++) {
    exceptionChecker.add(threadPool.submit(player));
}
exceptionChecker.forEach(future -> {
    try {
        future.get();
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
});
```

Your turn!

1. Look at the exercise2 package in ThreadingExercises
2. Check that the smoke test runs successfully
3. Modify the PlayGame code to allow two players to play at once (there are no turns in this game, everyone plays as fast as they can)
4. Check that the smoke test still runs successfully

Don't worry about any of the logic in AbstractGame for now

Problems with Threads

This code works just fine with a single thread

But what about with threads?

To the IDE!

```java
public class SpiderPlant {

    private int numChildren = 0;

    public SpiderPlant sproutChild() {
        numChildren++;
        return new SpiderPlant();
    }

    public int getNumChildren() {
        return numChildren;
    }
}
```

```java
@Test
public void testSpiderPlant() {
    SpiderPlant originalPlant = new SpiderPlant();
    List<SpiderPlant> childPlants = new ArrayList<>();

    for (int i = 0; i < 10; i++) {
        childPlants.add(originalPlant.sproutChild());
    }
    Assert.assertEquals(childPlants.size(), originalPlant.getNumChildren());
}
```

Problems with Threads

Heisenbugs: observing multithreaded code in the debugger changes the behavior of the code

When debugging multi-threaded code:

- Make it single-threaded
- Slowly shrink the section that is multi-threaded until you find the bug

This section of code is called a **critical section**, and must be protected from concurrent execution

```
public class SpiderPlant {

    private int numChildren = 0;

    public SpiderPlant sproutChild() {
        numChildren++;
        return new SpiderPlant();
    }

    public int getNumChildren() {
        return numChildren;
    }
}
```

Locking

A **lock** is a shared (on the heap!) object that only allows one thread to execute a piece of code at once

To execute the code, the thread **acquires** the lock, **holds** the lock while running, and then **releases** the lock

Lock Attributes:

re-entrant: If a thread already holds the lock, it can re-acquire the lock

Important to avoid **deadlock**

fair: threads receive the lock in roughly the order they request it

Synchronized

synchronized: Java keyword to protect code that must be single-threaded

- Not fair
- Re-entrant
- By default, lock is **per object,** but can specify any object

Synchronized

synchronized: Java keyword to protect code that must be single-threaded

Heuristic: always synchronize on the **shared** object

```
@Test
public void testSpiderPlantMultiThreaded() {
    SpiderPlant originalPlant = new SpiderPlant();
    List<SpiderPlant> childPlants = Collections.synchronizedList(new ArrayList<>());

    Callable<Void> createPlants = () -> {
        synchronized (originalPlant) {
            int nPlantsPerThread = 10;
            for (int i = 0; i < nPlantsPerThread; i++) {
                childPlants.add(originalPlant.sproutChild());
            }
            return null;
        }
    };
```

When In Doubt, Synchronize Broadly

Then binary-search to where the bug is

```
@Test
public void testSpiderPlantMultiThreaded()
    SpiderPlant originalPlant = new SpiderI
    List<SpiderPlant> childPlants = Collec

    Callable<Void> createPlants = () -> {
        synchronized (this) {
            int nPlantsPerThread = 10;
            for (int i = 0; i < nPlantsPer
                childPlants.add(originalPl
            }
            return null;
        }
    };
```

```
@Test
public void testSpiderPlantMultiThreaded() {
    SpiderPlant originalPlant = new SpiderPlant();
    List<SpiderPlant> childPlants = Collections.sy

    Callable<Void> createPlants = () -> {
        synchronized (TestSpiderPlant.class) {
            int nPlantsPerThread = 10;
            for (int i = 0; i < nPlantsPerThread;
                childPlants.add(originalPlant.spr
            }
            return null;
        }
    };
```

Synchronized on Methods

On a method, **synchronized** implicitly uses (this) as the lock object

This makes the method **atomic** (ie, single-threaded - it's run as one atomic, indivisible, block)

```
public synchronized SpiderPlant sproutChild() {
    numChildren++;
    return new SpiderPlant();
}
```

Race Condition

Threads are **racing** to get to the code, and they trip over each other

`numChildren = numChildren + 1`:

Thread:

1. Copy the value of numChildren into a register
2. Increment that value by 1
3. Write the result back to numChildren

Race Condition

`numChildren = numChildren + 1`: Assume numChildren = 4 to start

Thread:

1. Copy the value of numChildren into a register (register = 4)
2. Increment that value by 1 (register = 5)
3. Write the result back to numChildren (numChildren = 5)

Thread:

1. Copy the value of numChildren into a register (register = 4)
2. Increment that value by 1 (register = 5)
3. Write the result back to numChildren (numChildren = 5)

Your Turn!

1. Look at the exercise3 package in ThreadingExercises
2. Add `synchronized` blocks to fix the failing unit test (note: the test may not fail right away, you may have to run it a few times)
3. What's another way to fix this that doesn't involve `synchronized`?

Semaphore

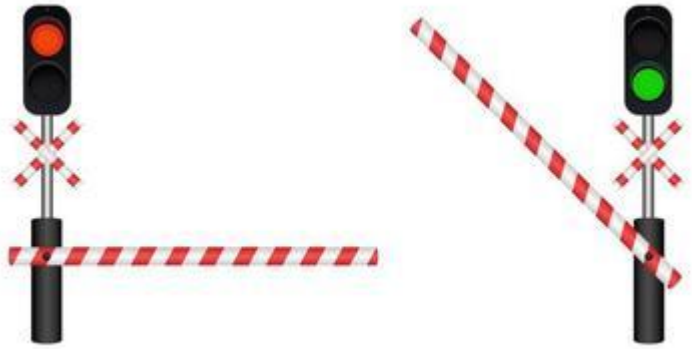
What if you want threads to take turns in a more organized fashion?

A **semaphore** allows some number of threads to run at once, but no more than that

Ex: a section of train track with only two tracks should only permit two trains at a time to enter

Supports **fair** rotation through waiting threads:

```
Semaphore s = new Semaphore(2, true);
```



CountDownLatch

Used to wait for several threads to all complete a task

Similar to using get on a Future

Ex: Everyone picks a card to play, and places it face down in front of them. Once everyone has picked a card, then everyone flips theirs over.

```
int numPlayers = 3;
List<Card> cardsToFlip = Collections.synchronizedList(new ArrayLis
CountDownLatch allCardsSelected = new CountDownLatch(numPlayers);

Callable<Void> cardSelector = () -> {
    cardsToFlip.add(selectCard());
    allCardsSelected.countDown();
    return null;
};

Callable<Void> cardFlipper = () -> {
    allCardsSelected.await();
    for (Card card : cardsToFlip) {
        flip(card);
    }
    return null;
};
```

Read/Write Locks

Two related locks - one allows for many threads to access something, one locks all threads out.

Ex: Reading vs writing a file

Write lock - identify under what circumstances code needs to be single threaded (generally, changing a value)

Read lock - add this around any code that can't run while the write lock is held

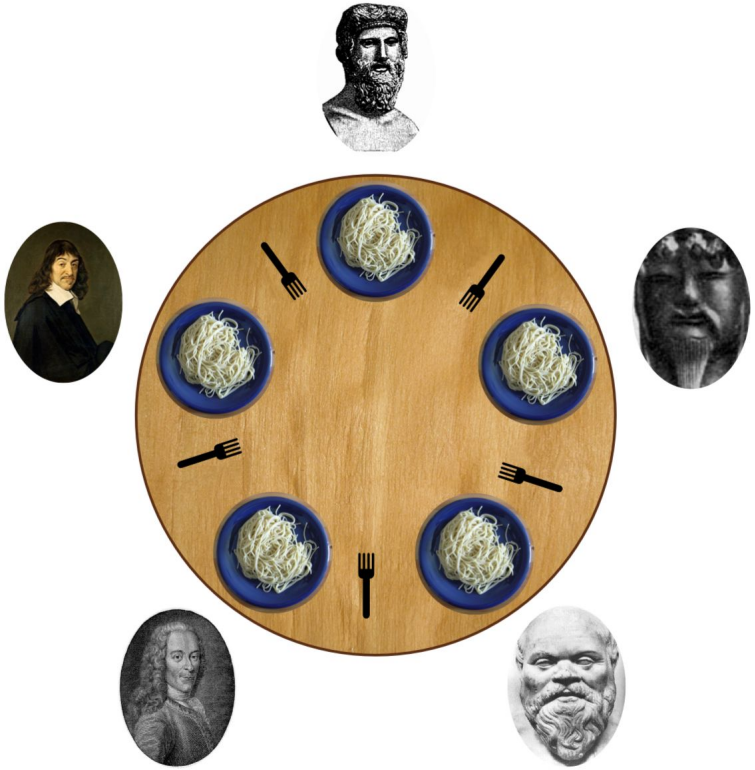
Note the try/finally - necessary whenever you're handling locking/unlocking

```
private ReadWriteLock deckLock = new ReentrantReadWriteLock();
private List<Card> deck = new ArrayList<>();

public Card lookAtTopCard() {
    try {
        deckLock.readLock().lock();
        return deck.get(0);
    } finally {
        deckLock.readLock().unlock();
    }
}

public void playCard(Card card) {
    try {
        deckLock.writeLock().lock();
        deck.add(0, card);
    } finally {
        deckLock.writeLock().unlock();
    }
}
```

Deadlock

Two (or more) threads are waiting on resources the other thread holds

No thread can make further progress, and the program grinds to a halt



Deadlock

First, threadOne acquires the FlipCards.class lock

Next, threadTwo acquires the notThreadSafe lock

Now they're stuck - neither thread can execute its next line of code until the other thread exits its synchronized block

⇒ For multi-threading tests, it's a good idea to include a **timeout**

```
Callable<Void> threadOne = () -> {
    synchronized (FlipCards.class) {
        synchronized (notThreadSafe) {
            notThreadSafe.add("Hello");
        }
    }
    return null;
};

Callable<Void> threadTwo = () -> {
    synchronized (notThreadSafe) {
        synchronized (FlipCards.class) {
            notThreadSafe.add("World");
        }
    }
    return null;
};
```

Deadlock

ReadWrite Locks are especially prone to this, even when they're mostly re-entrant:

Acquire read lock, then read lock - re-entrant; if this thread already has a read lock, no one else can hold a write lock, so it's always safe to grant the read lock

Acquire write lock, then write lock - re-entrant; if this thread already has a write lock, no one else can hold a read or write lock, so it's always safe to grant the write lock

Acquire write lock then read lock - that's fine, we know no other thread can be holding either the read or write lock

Acquire the read lock then the write lock - NOPE

Deadlock

ReadWrite Locks are especially prone to this:

Thread 1: "I would like to acquire the read lock"

ReadWrite Lock: "Sure, here you go!"

Thread 2: "I would like to acquire the read lock"

ReadWrite Lock: "Sure, here you go!"

Thread 1: "I would like to acquire the write lock"

ReadWrite Lock: "Sure thing, as soon as everyone releases the read lock"

Thread 2: "I would like to acquire the write lock"

ReadWrite Lock: "Sure thing, as soon as everyone releases the read lock"

Thread 2: …

Thread 1: …

ReadWrite Lock: …

Your Turn!

1. Look at the exercise4 package in ThreadingExercises
2. Fix the warning/TODO around the List
3. Update the code so that the output list is sorted in descending order (you can check by running the test)
    a. Note that there is also a race condition that causes some numbers to be skipped entirely! Fixing the ordering will fix this too
    b. It's ok if the factorials print to the console out-of-order

Hint: You'll need multiple locks

Producer/Consumer Pattern

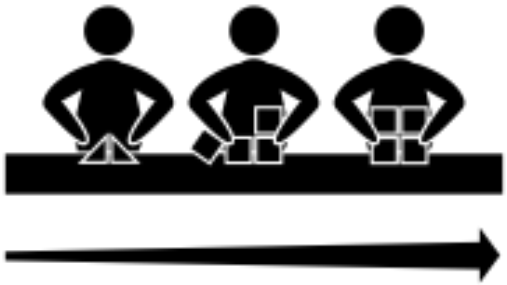
Design Patterns: Not just for single-threaded code!

Problem:

How to parallelize code that runs through a **pipeline?**

Solution: Use locks to handle passing partially completed jobs from one task to another

For ex: allows IO intensive computation at the same time as a CPU or network computation



Producer/Consumer Pattern

To set up the pipeline:

1: Set up how each stage will hand off to the next

Often, an **ArrayBlockingQueue**

Configure the queue with how many items can pile up while waiting to be processed

```
private static final int MAX_COUNTER_SPACE = 5;

public Collection<Cookie> bakeCookies(Queue<Flour> flour,
        Queue<Egg> eggs) {
    BlockingQueue<Dough> traysOnCounter =
            new ArrayBlockingQueue<Dough>(MAX_COUNTER_SPACE);
    List<Cookie> bakedCookies =
            Collections.synchronizedList(new ArrayList<>());
    AtomicBoolean allDoughMade = new AtomicBoolean(false);
```

Producer/Consumer Pattern

To set up the pipeline:

1: Set up how each stage will hand off to the next

Make sure to signal when no more work is coming

```
private static final int MAX_COUNTER_SPACE = 5;

public Collection<Cookie> bakeCookies(Queue<Flour> flour,
        Queue<Egg> eggs) {
    ArrayBlockingQueue<Dough> traysOnCounter =
            new ArrayBlockingQueue<Dough>(MAX_COUNTER_SPACE);
    List<Cookie> bakedCookies =
            Collections.synchronizedList(new ArrayList<>());
    AtomicBoolean allDoughMade = new AtomicBoolean(false);
```

Producer/Consumer Pattern

To set up the pipeline:

2: Set up each stage of the pipeline

- classic Callable we saw last week
- each Callable handles one stage

```
AtomicBoolean allDoughMade = new AtomicBoolean(false);

Callable<Void> mixer = () -> {
    while (!flour.isEmpty() && !eggs.isEmpty()) {
        Flour f = flour.remove();
        Egg e = eggs.remove();
        Dough dough = mix(f,e);
        traysOnCounter.offer(dough);
    }
    allDoughMade.set(true);
    return null;
};

Callable<Void> baker = () -> {
    while (!allDoughMade.get() || !traysOnCounter.isEmpty()) {
        Dough dough = traysOnCounter.poll(5, TimeUnit.SECONDS);
        if (dough != null) {
            Cookie cookie = bake(dough);
            bakedCookies.add(cookie);
        }
    }
    return null;
};
```

Producer/Consumer Pattern

To set up the pipeline:

3: Coordinate the handoff between stages

offer - similar to put/add/enqueue, except that this will **block** until space is available

Why?

- Memory pressure
- More evenly distribute CPU/disk/etc among stages

```
AtomicBoolean allDoughMade = new AtomicBoolean(false);

Callable<Void> mixer = () -> {
    while (!flour.isEmpty() && !eggs.isEmpty()) {
        Flour f = flour.remove();
        Egg e = eggs.remove();
        Dough dough = mix(f,e);
        traysOnCounter.offer(dough);
    }
    allDoughMade.set(true);
    return null;
};

Callable<Void> baker = () -> {
    while (!allDoughMade.get() || !traysOnCounter.isEmpty()) {
        Dough dough = traysOnCounter.poll(5, TimeUnit.SECONDS);
        if (dough != null) {
            Cookie cookie = bake(dough);
            bakedCookies.add(cookie);
        }
    }
    return null;
};
```

Producer/Consumer Pattern

To set up the pipeline:

3: Coordinate the handoff between stages

poll - similar to get/remove/dequeue, except that this will **block** until an element is available

Tricky part: what if no more dough is going to be made?

```
AtomicBoolean allDoughMade = new AtomicBoolean(false);

Callable<Void> mixer = () -> {
    while (!flour.isEmpty() && !eggs.isEmpty()) {
        Flour f = flour.remove();
        Egg e = eggs.remove();
        Dough dough = mix(f,e);
        traysOnCounter.offer(dough);
    }
    allDoughMade.set(true);
    return null;
};

Callable<Void> baker = () -> {
    while (!allDoughMade.get() || !traysOnCounter.isEmpty()) {
        Dough dough = traysOnCounter.poll(5, TimeUnit.SECONDS);
        if (dough != null) {
            Cookie cookie = bake(dough);
            bakedCookies.add(cookie);
        }
    }
    return null;
};
```

Producer/Consumer Pattern

To set up the pipeline:

3: Coordinate the handoff between stages

Always include a **timeout** with poll

- previous stage fails
- previous stage may not immediately know when it's done (filter step, waiting on a previous stage)

```
AtomicBoolean allDoughMade = new AtomicBoolean(false);

Callable<Void> mixer = () -> {
    while (!flour.isEmpty() && !eggs.isEmpty()) {
        Flour f = flour.remove();
        Egg e = eggs.remove();
        Dough dough = mix(f,e);
        traysOnCounter.offer(dough);
    }
    allDoughMade.set(true);
    return null;
};

Callable<Void> baker = () -> {
    while (!allDoughMade.get() || !traysOnCounter.isEmpty()) {
        Dough dough = traysOnCounter.poll(5, TimeUnit.SECONDS);
        if (dough != null) {
            Cookie cookie = bake(dough);
            bakedCookies.add(cookie);
        }
    }
    return null;
};
```

Producer/Consumer Pattern

Wait for the pipeline to finish

- Future.get() will **block** until the task completes
- If a stage fails, make sure to still inform subsequent stages

```
threadPool.shutdown();

try {
    mixerFuture.get();
} catch (Exception e) {
    allDoughMade.set(true);
}
try {
    bakerFuture.get();
} catch (Exception e) {
    throw new RuntimeException(e);
}

return bakedCookies;
```

Producer/Consumer Pattern

Slightly better error handling:

try must be combined with at least one of:

- try-with-resources, ex: `try (FileWriter f = new FileWriter("out.txt")) {}`
- **catch** which includes exception handling
- **finally** which runs some very short code whether an exception is thrown or not

```
Callable<Void> mixer = () -> {
    try {
        while (!flour.isEmpty() && !eggs.isEmpty()) {
            Flour f = flour.remove();
            Egg e = eggs.remove();
            Dough dough = mix(f,e);
            traysOnCounter.offer(dough);
        }
    } finally {
        allDoughMade.set(true);
    }
    return null;
};
```

Producer/Consumer Pattern

Risk with finally/catch blocks:

Throwing an exception **within** the finally block

- obscures the original error
- may not handle all cleanup properly

Often untested in the failure case

- NPEs are common

```
Callable<Void> mixer = () -> {
    try {
        while (!flour.isEmpty() && !eggs.isEmpty()) {
            Flour f = flour.remove();
            Egg e = eggs.remove();
            Dough dough = mix(f,e);
            traysOnCounter.offer(dough);
        }
    } finally {
        allDoughMade.set(true);
    }
    return null;
};
```

Your Turn!

1. Look at the exercise5 package in ThreadingExercises
2. Change the ExerciseSolver to a parallel pipeline with 2 stages; assume you can queue up 10 exercises to be solved at a time
3. Verify that both tests in TestExerciseSolver pass; initially, only one should pass, but they take a few seconds to run

Interrupts

What if you unexpectedly want to stop a Callable early?

The **interrupt** flag on a thread tracks whether another thread has asked this one to stop

Caution: generally **checking** whether a thread is interrupted also **clears** this flag:

```
Thread.interrupted()
```

= 'have I been interrupted since the last time I asked?'

Interrupts

To tell a thread to stop early, use

`Future.cancel(true)`

- keep the Callable from ever running if it hasn't started yet
- set the **interrupt** flag if the Callable has started

Note: any calls to Future.get will **immediately** throw a CancellationException

- to wait for the thread to actually cancel, you need a lock/shared state

```
Callable<Void> makeJoke = () -> {
    while (true) {
        System.out.println("Knock knock");
        System.out.println("Who's there?");
        System.out.println("Interrupting cow");
        System.out.print("Interrupting cow");
        if (Thread.interrupted()) {
            break;
        }
        System.out.println(" who?");
    }
    return null;
};

ExecutorService threadPool = Executors.newCachedThreadPool();
Future<Void> jokeFuture = threadPool.submit(makeJoke);
Thread.sleep(100);
jokeFuture.cancel(true);
System.out.println("-- Moo!");
```

Interrupts

What things check the interrupt flag?

- Most **blocking** calls
- Anything that throws InterruptedException

One option if you don't want to re-throw an InterruptedException:

```
catch (InterruptedException) {
   Thread.interrupt();
}
```

Let's Review: Threading

Stack vs Heap - what data goes where

- Translating code to a sketch of what memory looks like
- How are pointers configured
- Pass by Reference vs Pass by Value for parameters

Threading Libraries: What are 3 major classes that you would use for handling threads in java?

What is: a deadlock, a critical section of code, and a race condition?

What does the synchronized keyword do, and how do you use it?

What are 3 useful classes for handling locking?